



SCHOOL OF COMPUTER TECHNOLOGY

AASD 4008 Big Data Tools and Technologies II

What is PySpark?

PySpark is the Python API for Apache Spark. It enables you to perform **real-time, large-scale data processing** in a **distributed environment** using **Python**.

PySpark combines Python's learnability and ease of use with the power of Apache Spark to enable processing and analysis of data at any size for everyone familiar with Python.



What is PySpark - Continued

PySpark is the Python API for Apache Spark, an open source, distributed computing framework and set of libraries for real-time, large-scale data processing. If you're already familiar with Python and libraries such as Pandas, then PySpark is a good language to learn to create more scalable analyses and pipelines.

Apache Spark is basically a computational engine that works with huge sets of data by processing them in parallel and batch systems. Spark is written in Scala, and PySpark was released to support the collaboration of Spark and Python

What is PySpark?



PySpark supports all of Spark's features such as Spark SQL, DataFrames, Structured Streaming, Machine Learning (MLlib) and Spark Core.

Spark SQL and
DataFrames

Pandas API on
Spark

Structured
Streaming

Machine
Learning
MLlib

Spark Core and RDDs

Spark SQL and DataFrames



Spark SQL is Apache Spark's module for working with structured data. It allows you to seamlessly mix SQL queries with Spark programs. With PySpark DataFrames you can efficiently read, write, transform, and analyze data using Python and SQL. Whether you use Python or SQL, the same underlying execution engine is used so you will always leverage the full power of Spark.



Spark SQL and DataFrames



```
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("SalesAnalysis").getOrCreate()

# Load the CSV data into a DataFrame
sales_data = spark.read.csv("sales_data.csv", header=True, inferSchema=True)

# Register the DataFrame as a temporary table
sales_data.createOrReplaceTempView("sales_table")

# Write and execute the Spark SQL code
result = spark.sql("""
    SELECT
        product_category,
        SUM(sales_amount) AS total_sales
    FROM
        sales_table
    GROUP BY
        product_category
""")

# Show the result
result.show()
```

Pandas API on Spark



Pandas API on Spark allows you to scale your pandas workload to any size by running it distributed across multiple nodes.

If you are already familiar with pandas and want to leverage Spark for big data, pandas API on Spark makes you immediately productive and lets you migrate your applications without modifying the code.

Pandas API on Spark



```
import pandas as pd

# Create a Spark session
spark = SparkSession.builder.appName("PandasSparkExample").getOrCreate()

# Sample data
data = {
    'product_id': [1, 2, 3, 4, 5, 6],
    'product_category': ['Electronics', 'Electronics', 'Home Appliances', 'Home Appliances', 'Clothing', 'Clothing'],
    'sales_amount': [500, 700, 300, 200, 100, 150]
}

# Create a PySpark DataFrame
df = spark.createDataFrame(pd.DataFrame(data))

# Use PySpark SQL functions to calculate total sales
result_df = df.groupBy("product_category").agg(F.sum("sales_amount").alias("total_sales"))

# Convert PySpark DataFrame to Pandas DataFrame
result_pandas = result_df.toPandas()

# Show the result
print(result_pandas)

# Stop the Spark session
spark.stop()
```


Structured Streaming



Structured Streaming is a **scalable and fault-tolerant** stream processing engine built on the Spark SQL engine. You can express your streaming computation the same way you would express a batch computation on static data.

The Spark SQL engine will take care of running it incrementally and continuously and updating the final result as streaming data continues to arrive.

Machine Learning



Built on top of **Spark**, **MLlib** is a scalable machine learning library that provides a uniform set of high-level APIs that help users create and tune practical machine learning pipelines.

Machine Learning



```
from pyspark.sql import SparkSession
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator

# Create a Spark session
spark = SparkSession.builder.appName("MLlibPipelineExample").getOrCreate()

# Load the Iris dataset
iris_data = spark.read.csv("iris.csv", header=True, inferSchema=True)

# Prepare features and label columns
feature_columns = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
assembler = VectorAssembler(inputCols=feature_columns, outputCol="features")
indexer = StringIndexer(inputCol="species", outputCol="label")

# Define a Logistic Regression model
lr = LogisticRegression(maxIter=10, regParam=0.01)

# Create a pipeline
pipeline = Pipeline(stages=[assembler, indexer, lr])

# Split the data into training and testing sets
train_data, test_data = iris_data.randomSplit([0.8, 0.2], seed=123)

# Train the pipeline
model = pipeline.fit(train_data)

# Make predictions on the test set
predictions = model.transform(test_data)

# Evaluate the model
evaluator = BinaryClassificationEvaluator(labelCol="label")
auc = evaluator.evaluate(predictions)
print(f"Area Under ROC Curve (AUC): {auc}")

# Stop the Spark session
spark.stop()
```

Spark Core and RDDs



Spark Core is the underlying general execution engine for the Spark platform that all other functionality is built on top of. It provides RDDs (Resilient Distributed Datasets) and in-memory computing capabilities.

Note that the RDD API is a low-level API which can be difficult to use and you do not get the benefit of Spark's automatic query optimization capabilities. **We recommend using DataFrames instead of RDDs** as it allows you to express what you want more easily and lets Spark automatically construct the most efficient query for you.

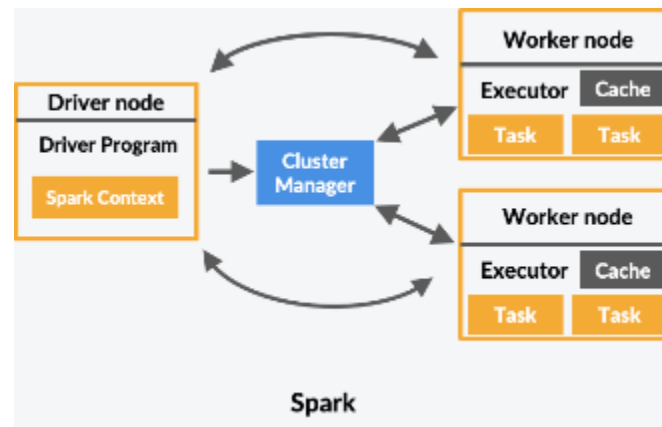
Motivation



The motivation behind choosing **PySpark** lies in the widespread popularity of Python in data science and analytics. The seamless integration with existing Python libraries and the ease of expressing complex data processing tasks in Python make PySpark a compelling choice.

Its user-friendly syntax and extensive community support contribute to its growing adoption in the big data landscape.

Architecture



PySpark's architecture revolves around three main components: the **Driver Program**, **Cluster Manager**, and **Executors**. The core data abstraction is the Resilient Distributed Dataset (RDD), which enables fault-tolerant parallel processing. The Directed Acyclic Graph (DAG) execution model governs the flow of data transformations, ensuring efficient and scalable computation in a distributed environment.

Usage



PySpark finds application in various domains, serving as a robust solution for data processing and analysis at scale. Its flexibility is evident in tasks such as machine learning model training and deployment, real-time data streaming, and integration with different data sources. PySpark empowers organizations to handle large volumes of data efficiently and make informed decisions.

Pyspark in Data Science



Data scientists benefit significantly from PySpark, as it provides a powerful tool for handling large datasets in a distributed environment. The seamless integration with popular Python libraries such as Pandas and NumPy enhances the data science workflow. PySpark's capabilities make it an attractive choice for data scientists working on complex analytics projects.

Business Cases



Numerous real-world examples showcase the impact of PySpark in improving performance and efficiency in big data processing. Companies across various industries leverage PySpark to derive insights from vast datasets, ultimately leading to better decision-making. Its cost-effective solutions make it an integral part of the big data landscape for businesses seeking scalable and powerful data processing.

Apache Spark, including its Python API PySpark, is widely used in various industries for big data processing and analytics. Here are three real-life business use cases where PySpark is applied

Business Cases – continued



Large-scale Data Processing in E-commerce:

Problem: E-commerce platforms generate vast amounts of data, including customer interactions, transaction histories, and clickstream data.

Solution: PySpark can be employed to process and analyze this massive volume of data efficiently. For example, businesses can use PySpark to identify patterns in customer behavior, optimize product recommendations, and perform real-time analysis of website traffic. This helps in improving customer experience, increasing sales, and optimizing supply chain management.

Financial Fraud Detection in Banking:

Problem: Financial institutions face challenges in detecting fraudulent activities in real-time due to the large volume and complexity of financial transactions.

Solution: PySpark can be utilized for building scalable and distributed fraud detection systems. By processing and analyzing large datasets in parallel, PySpark enables the identification of patterns indicative of fraud. Machine learning models integrated with PySpark can continuously learn and adapt to new fraud techniques, providing real-time detection and prevention of fraudulent transactions, enhancing the security of financial systems.

Healthcare Analytics for Patient Care:

Problem: Healthcare organizations deal with massive datasets containing patient records, medical histories, and diagnostic information. Analyzing this data for insights is challenging with traditional tools.

Solution: PySpark can be applied to process and analyze healthcare data efficiently. For instance, it can be used to predict patient outcomes, optimize resource allocation in hospitals, and identify trends in disease prevalence. PySpark's ability to handle large-scale data processing makes it suitable for building predictive models that assist healthcare professionals in making informed decisions, improving patient care, and optimizing hospital operations.

PySpark – Structured – CSV Data

Reading data from **CSV files** in PySpark involves using the **SparkSession** and its capabilities to handle distributed data processing. The process typically begins by creating a Spark session, a unified entry point for reading data in PySpark.

```
# Import required modules
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# Read CSV file into a DataFrame
csv_file_path = "path/to/your/file.csv"
df = spark.read.csv(csv_file_path, header=True, inferSchema=True)

# Show the DataFrame
df.show()
```

PySpark – Structured – Database



Reading data from a **database** in PySpark involves utilizing the SparkSession and the read module, specifically the **jdbc** method, to establish a connection and fetch data. Here's a concise example for reading from a database:

```
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# Database connection properties
database_url = "jdbc:postgresql://your_database_url:your_port/your_database"
database_properties = {
    "user": "your_username",
    "password": "your_password",
    "driver": "org.postgresql.Driver"
}

# Define the query to fetch data
query = "SELECT * FROM your_table"

# Read data from the database into a DataFrame
df = spark.read.jdbc(url=database_url, table=query, properties=database_properties)

# Show the DataFrame
df.show()
```

PySpark – JSON Data



Reading data from JSON files in PySpark is a common task that can be accomplished using the SparkSession and its read module. Here's a brief example:

```
C / courses / CSI / 2024-02 - bigdata tools and tech II / spark / pyspark / samplecode / 1_CS
# Import required modules
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("example").getOrCreate()

# Define the path to the JSON file
json_file_path = "path/to/your/file.json"

# Read JSON file into a DataFrame
df = spark.read.json(json_file_path)

# Show the DataFrame
df.show()
```